Математическое моделирование

Лабораторная работа № 5

Элсаиед Адел

Содержание

1 Цель работы **6**

2 Задание **7**

3 Выполнение лабораторной работы **8**
3.1 Теоретические сведения 8
3.2 Задача 9

4 Модель хищник-жертва: численное решение и визуализация **10**
4.1 Инициализация проекта и загрузка пакетов 10
4.2 Начальные условия и временной интервал 11
4.3 Параметры модели 11
4.4 Определение модели 11
4.5 Утилита: один прогон (решение, таблица, графики, сохранение) . 12
4.6 Запуск модели 14
4.7 Вывод первых строк таблицы 14
4.8 Показать график в интерактивной среде 14

5 Параметрическое исследование модели хищник-жертва **15**
5.1 Активация проекта и загрузка пакетов 15
5.2 Установка каталогов 15
5.3 Определение моделей 16
5.4 Базовые параметры в Dict 16
5.5 Функция-обертка для запуска одного эксперимента 17
5.6 Запуск базового эксперимента 20
5.7 Визуализация базового эксперимента 21
5.8 Фазовый портрет базового эксперимента 21
5.9 Второй базовый эксперимент (расширенная система) 22
5.10 Параметрическое сканирование 24
5.11 Запуск всех экспериментов и сбор результатов 25
5.12 Анализ результатов сканирования 27
5.13 Сравнительный график x(t) 27
5.14 Сравнительный график y(t) 29
5.15 Сравнительный фазовый портрет 30
5.16 График зависимости norm_final от параметров 31
5.17 Бенчмаркинг 32

5.18 График времени вычисления 34
5.19 Сохранение всех результатов 35
5.20 Базовые эксперименты 36
5.21 Параметрическое сканирование 40
5.22 Анализ метрики norm_final 43
5.23 Время вычислений 44
5.24 Выводы 45

Список литературы **46**

Список иллюстраций

5.1 Базовая модель: зависимость x(t), y(t) 36
5.2 Базовая модель: фазовый портрет 37
5.3 Расширенная модель: зависимость x(t), y(t) 38
5.4 Расширенная модель: фазовый портрет 39
5.5 Сканирование траекторий x(t) 40
5.6 Сканирование траекторий y(t) 41
5.7 Сканирование фазовых траекторий 42
5.8 Зависимость norm_final 43
5.9 Зависимость времени вычисления 44

Список таблиц

1 Цель работы

Изучить модель хищник-жертва

2 Задание

1. Построить график зависимости x от y и графики функций $x(t)$, $y(t)$
2. Найти стационарное состояние системы

3 Выполнение лабораторной работы

3.1 Теоретические сведения

В данной лабораторной работе рассматривается математическая модель системы «Хищник-жертва».

Рассмотрим базисные компоненты системы. Пусть система имеет X хищников и Y жертв. И пусть для этой системы выполняются следующие предположения: (Модель Лотки-Вольтерра) 1. Численность популяции жертв и хищников зависят только от времени (модель не учитывает пространственное распределение популяции на занимаемой территории) 2. В отсутствии взаимодействия численность видов изменяется по модели Мальтуса, при этом число жертв увеличивается, а число хищников падает 3. Естественная смертность жертвы и естественная рождаемость хищника считаются несущественными 4. Эффект насыщения численности обеих популяций не учитывается 5. Скорость роста численности жертв уменьшается пропорционально численности хищников:

$$\begin{cases} \frac{dx}{dt} = (-ax(t) + by(t)x(t)) \\ \frac{dy}{dt} = (cy(t) - dy(t)x(t)) \end{cases}$$

Параметр a определяет коэффициент смертности хищников, b – коэффициент естественного прироста хищников, c – коэффициент прироста жертв и d – коэффициент смертности жертв

В зависимости от этих параметрах система и будет изменяться. Однако сле-

дует выделить одно важное состояние системы, при котором не происходит никаких изменений как со стороны хищников, так и со стороны жертв. Это, так называемое, стационарное состояние системы. При нем, как уже было отмечено, изменение численности популяции равно нулю. Следовательно, при отсутствии изменений в системе $\frac{dx}{dt} = 0, \frac{dy}{dt} = 0$

Пусть по условию есть хотя бы один хищник и хотя бы одна жертва: $x > 0, y > 0$ Тогда стационарное состояние системы определяется следующим образом:

$$x_0 = \frac{a}{b}, y_0 = \frac{c}{d}$$

3.2 Задача

Для модели «хищник-жертва»:

$$\begin{cases} \frac{dx}{dt} = -0.25x(t) + 0.025y(t)x(t) \\ \frac{dy}{dt} = 0.45y(t) - 0.045y(t)x(t) \end{cases}$$

Постройте график зависимости численности хищников от численности жертв, а также графики изменения численности хищников и численности жертв при следующих начальных условиях: $x_0 = 8, y_0 = 11$ Найдите стационарное состояние системы

Стационарное состояние $x_0 = \frac{a}{b} = 10, y_0 = \frac{c}{d} = 10$

Для моделирования процесса и построения графиков использовались внешние файлы с программным кодом:

4 Модель хищник-жертва: численное решение и визуализация

Цель: решить систему ОДУ Лотки–Вольтерры, построить графики изменения популяций во времени, построить фазовый портрет, сохранить изображения и таблицу с результатами.

4.1 Инициализация проекта и загрузка пакетов

```
using DrWatson
@quickactivate "project"

using DifferentialEquations
using Plots
using DataFrames
using CSV
using JLD2

script_name = isempty(PROGRAM_FILE) ? "interactive" : splitext(basename(PROGRAM_FILE)
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

4.2 Начальные условия и временной интервал

```
x0 = 8.0
y0 = 11.0

u0 = [x0, y0]

t0 = 0.0
tmax = 150.0
tspan = (t0, tmax)
```

4.3 Параметры модели

```
a = 0.25
b = 0.025
c = 0.45
d = 0.045

p = (a = a, b = b, c = c, d = d)
```

4.4 Определение модели

x(t) — численность жертв y(t) — численность хищников

dx/dt = -ax + bxy dy/dt = cy - dxy

```
function predator_prey!(dy, y, p, t)
    dy[1] = -p.a * y[1] + p.b * y[1] * y[2]
```

```
    dy[2] =  p.c * y[2] - p.d * y[1] * y[2]
end
```

4.5 Утилита: один прогон (решение, таблица, графики, сохранение)

```
function run_case(case_name; model!, u0, tspan, p, nt=1000)
```

— Сетка по времени —

```
    tgrid = collect(LinRange(tspan[1], tspan[2], nt))
```

— Решение ОДУ —

```
    prob = ODEProblem(model!, u0, tspan, p)
    sol = solve(prob, Tsit5(), saveat=tgrid)
```

— Таблица с результатами —

```
    df = DataFrame(
        t = sol.t,
        x = first.(sol.u),
        y = last.(sol.u)
    )
```

— График x(t) и y(t) —

```
    plt_time = plot(
        sol,
        xlabel = "t",
```

```
        ylabel = "Численность",
        label = ["x(t) — жертвы" "y(t) — хищники"],
        lw = 2,
        title = "Динамика популяций — $case_name",
        legend = :topright
    )
```

— Фазовый портрет —

```
    plt_phase = plot(
        sol,
        idxs = (1, 2),
        xlabel = "x",
        ylabel = "y",
        lw = 2,
        label = "Фазовая траектория",
        title = "Фазовый портрет — $case_name",
        legend = :topright
    )
```

— Сохранение графиков —

```
    savefig(plt_time, plotsdir(script_name, "$(case_name)_time.png"))
    savefig(plt_phase, plotsdir(script_name, "$(case_name)_phase.png"))
```

— Сохранение таблицы —

```
    CSV.write(datadir(script_name, "table_$(case_name).csv"), df)
```

— Сохранение данных в JLD2 —

```
    @save datadir(script_name, "data_$(case_name).jld2") df p u0 tspan nt

    return (sol = sol, df = df, plt_time = plt_time, plt_phase = plt_phase)
end
```

4.6 Запуск модели

```
res = run_case(
    "predator_prey";
    model! = predator_prey!,
    u0 = u0,
    tspan = tspan,
    p = p,
    nt = 1000
)
```

4.7 Вывод первых строк таблицы

```
println("Первые 5 строк таблицы результатов:")
println(first(res.df, 5))
```

4.8 Показать график в интерактивной среде

```
res.plt_time
```

5 Параметрическое исследование модели хищник-жертва

5.1 Активация проекта и загрузка пакетов

```
using DrWatson
@quickactivate "project"

using DifferentialEquations
using DataFrames
using Plots
using JLD2
using BenchmarkTools
using CSV
```

5.2 Установка каталогов

```
script_name = isempty(PROGRAM_FILE) ? "interactive" : splitext(basename(PROGRAM_FILE)
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

5.3 Определение моделей

Первая система (базовая модель Лотки–Вольтерры): dx/dt = -a*x* + *b*x*y* *dy/dt* = *c*y - d*x*y

```
function syst_base!(dy, y, p, t)
    dy[1] = -p.a * y[1] + p.b * y[1] * y[2]
    dy[2] =  p.c * y[2] - p.d * y[1] * y[2]
end
```

Вторая система (усложнённая модель с внутривидовым ограничением жертв): dx/dt = -a*x* + *b*x*y* - *k*x^2 dy/dt = c*y* - *d*x*y

```
function syst_extended!(dy, y, p, t)
    dy[1] = -p.a * y[1] + p.b * y[1] * y[2] - p.k * y[1]^2
    dy[2] =  p.c * y[2] - p.d * y[1] * y[2]
end
```

5.4 Базовые параметры в Dict

model_type управляет выбором системы: :base -> классическая модель :extended -> модель с дополнительным нелинейным членом -k*x^2

```
base_params = Dict(
    :x0 => 8.0,
    :y0 => 11.0,

    :tspan => (0.0, 150.0),
    :nt => 1000,

    :model_type => :base,      # :base или :extended
```

Параметры базовой модели

```
    :a => 0.25,
    :b => 0.025,
    :c => 0.45,
    :d => 0.045,
```

Дополнительный параметр расширенной модели

```
    :k => 0.002,

    :solver => Tsit5(),
    :experiment_name => "base_experiment"
)

println("Базовые параметры эксперимента:")
for (key, value) in base_params
    println(" $key = $value")
end
```

5.5 Функция-обертка для запуска одного эксперимента

```
function run_single_experiment(params::Dict)
    x0 = params[:x0]
    y0 = params[:y0]
    tspan = params[:tspan]
    nt = params[:nt]
```

```julia
    model_type = params[:model_type]
    a = params[:a]
    b = params[:b]
    c = params[:c]
    d = params[:d]
    k = params[:k]
    solver = params[:solver]

    u0 = [x0, y0]
    tgrid = collect(LinRange(tspan[1], tspan[2], nt))

    if model_type == :base
        p = (a = a, b = b, c = c, d = d)
        prob = ODEProblem(syst_base!, u0, tspan, p)
    else
        p = (a = a, b = b, c = c, d = d, k = k)
        prob = ODEProblem(syst_extended!, u0, tspan, p)
    end

    sol = solve(prob, solver; saveat = tgrid)

    x_vals = first.(sol.u)
    y_vals = last.(sol.u)
```

— Мини-анализ —

```julia
    x_final = x_vals[end]
    y_final = y_vals[end]
```

```
    x_max = maximum(x_vals)
    y_max = maximum(y_vals)

    x_min = minimum(x_vals)
    y_min = minimum(y_vals)

    x_mean = mean(x_vals)
    y_mean = mean(y_vals)

    norm_final = sqrt(x_final^2 + y_final^2)

    return Dict(
        "solution" => sol,
        "t_points" => sol.t,
        "x_values" => x_vals,
        "y_values" => y_vals,

        "x_final" => x_final,
        "y_final" => y_final,
        "x_max" => x_max,
        "y_max" => y_max,
        "x_min" => x_min,
        "y_min" => y_min,
        "x_mean" => x_mean,
        "y_mean" => y_mean,
        "norm_final" => norm_final,

        "parameters" => params
```

```
    )
end
```

5.6 Запуск базового эксперимента

```
data_base, path_base = produce_or_load(
    datadir(script_name, "single"),
    base_params,
    run_single_experiment;
    prefix = "ode",
    tag = false,
    verbose = true
)

println("\nРезультаты базового эксперимента:")
println(" x_final: ", data_base["x_final"])
println(" y_final: ", data_base["y_final"])
println(" x_max: ", data_base["x_max"])
println(" y_max: ", data_base["y_max"])
println(" x_min: ", data_base["x_min"])
println(" y_min: ", data_base["y_min"])
println(" x_mean: ", round(data_base["x_mean"]; digits = 4))
println(" y_mean: ", round(data_base["y_mean"]; digits = 4))
println(" norm_final: ", round(data_base["norm_final"]; digits = 4))
println(" Файл результатов: ", path_base)
```

5.7 Визуализация базового эксперимента

```
p1 = plot(
    data_base["t_points"], data_base["x_values"],
    label = "x(t) — жертвы",
    xlabel = "t",
    ylabel = "Численность",
    title = "Базовый эксперимент (model_type=$(base_params[:model_type]))",
    lw = 2,
    legend = :topright,
    grid = true
)
plot!(
    p1,
    data_base["t_points"], data_base["y_values"],
    label = "y(t) — хищники",
    lw = 2
)

savefig(p1, plotsdir(script_name, "single_experiment_base.png"))
```

5.8 Фазовый портрет базового эксперимента

```
p2 = plot(
    data_base["x_values"], data_base["y_values"],
    xlabel = "x",
    ylabel = "y",
```

```
    label = "Фазовая траектория",
    title = "Фазовый портрет (model_type=$(base_params[:model_type]))",
    lw = 2,
    legend = :topright,
    grid = true
)

savefig(p2, plotsdir(script_name, "single_experiment_base_phase.png"))
```

5.9 Второй базовый эксперимент (расширенная система)

```
base_params2 = copy(base_params)
base_params2[:model_type] = :extended
base_params2[:experiment_name] = "base_experiment_extended"

data_base2, path_base2 = produce_or_load(
    datadir(script_name, "single"),
    base_params2,
    run_single_experiment;
    prefix = "ode",
    tag = false,
    verbose = true
)

println("\nРезультаты второго базового эксперимента:")
println(" x_final: ", data_base2["x_final"])
```

```
println(" y_final: ", data_base2["y_final"])
println(" x_max: ", data_base2["x_max"])
println(" y_max: ", data_base2["y_max"])
println(" x_min: ", data_base2["x_min"])
println(" y_min: ", data_base2["y_min"])
println(" x_mean: ", round(data_base2["x_mean"]; digits = 4))
println(" y_mean: ", round(data_base2["y_mean"]; digits = 4))
println(" norm_final: ", round(data_base2["norm_final"]; digits = 4))
println(" Файл результатов: ", path_base2)

p3 = plot(
    data_base2["t_points"], data_base2["x_values"],
    label = "x(t) — жертвы",
    xlabel = "t",
    ylabel = "Численность",
    title = "Базовый эксперимент (model_type=$(base_params2[:model_type]))",
    lw = 2,
    legend = :topright,
    grid = true
)
plot!(
    p3,
    data_base2["t_points"], data_base2["y_values"],
    label = "y(t) — хищники",
    lw = 2
)

savefig(p3, plotsdir(script_name, "single_experiment_extended.png"))
```

```
p4 = plot(
    data_base2["x_values"], data_base2["y_values"],
    xlabel = "x",
    ylabel = "y",
    label = "Фазовая траектория",
    title = "Фазовый портрет (model_type=$(base_params2[:model_type]))",
    lw = 2,
    legend = :topright,
    grid = true
)

savefig(p4, plotsdir(script_name, "single_experiment_extended_phase.png"))
```

5.10 Параметрическое сканирование

Сканируем: - для базовой модели параметр a - для расширенной модели параметр k

```
param_grid = Dict(
    :x0 => [8.0],
    :y0 => [11.0],

    :tspan => [(0.0, 150.0)],
    :nt => [1000],

    :model_type => [:base, :extended],

    :a => [0.15, 0.25, 0.35, 0.50],
```

```
    :b => [0.025],
    :c => [0.45],
    :d => [0.045],

    :k => [0.0, 0.001, 0.002, 0.005],

    :solver => [Tsit5()],
    :experiment_name => ["parametric_scan"]
)

all_params = dict_list(param_grid)

println("\n" * "="^60)
println("ПАРАМЕТРИЧЕСКОЕ СКАНИРОВАНИЕ")
println("Всего комбинаций параметров: ", length(all_params))
println("Исследуемые model_type: ", param_grid[:model_type])
println("Исследуемые a: ", param_grid[:a])
println("Исследуемые k: ", param_grid[:k])
println("="^60)
```

5.11 Запуск всех экспериментов и сбор результатов

```
all_results = []
all_dfs = []

for (i, params) in enumerate(all_params)
    println("Прогресс: $i/$(length(all_params)) | model_type=$(params[:model_type]) |
```

```
data, path = produce_or_load(
    datadir(script_name, "parametric_scan"),
    params,
    run_single_experiment;
    prefix = "scan",
    tag = false,
    verbose = false
)

result_summary = merge(
    params,
    Dict(
        :x_final => data["x_final"],
        :y_final => data["y_final"],
        :x_max => data["x_max"],
        :y_max => data["y_max"],
        :x_min => data["x_min"],
        :y_min => data["y_min"],
        :x_mean => data["x_mean"],
        :y_mean => data["y_mean"],
        :norm_final => data["norm_final"],
        :filepath => path
    )
)
push!(all_results, result_summary)

df = DataFrame(
    t = data["t_points"],
```

```
        x = data["x_values"],
        y = data["y_values"],
        model_type = fill(string(params[:model_type]), length(data["t_points"])),
        a = fill(params[:a], length(data["t_points"])),
        k = fill(params[:k], length(data["t_points"]))
    )
    push!(all_dfs, df)
end
```

5.12 Анализ результатов сканирования

```
results_df = DataFrame(all_results)

println("\nСводная таблица результатов (первые 10 строк):")
println(first(results_df, 10))

CSV.write(datadir(script_name, "results_summary.csv"), results_df)
```

5.13 Сравнительный график x(t)

```
p5 = plot(size = (950, 520), dpi = 150)
for params in all_params
    data, _ = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_single_experiment;
```

```
        prefix = "scan",
        tag = false,
        verbose = false
    )

    label_text = params[:model_type] == :base ?
        "base, a=$(params[:a])" :
        "extended, k=$(params[:k])"

    plot!(
        p5,
        data["t_points"], data["x_values"],
        label = label_text,
        lw = 2,
        alpha = 0.8
    )
end
plot!(
    p5,
    xlabel = "t",
    ylabel = "x(t)",
    title = "Сканирование: траектории x(t) для разных параметров",
    legend = :outerright,
    grid = true
)
savefig(p5, plotsdir(script_name, "parametric_scan_x_comparison.png"))
```

5.14 Сравнительный график y(t)

```
p6 = plot(size = (950, 520), dpi = 150)
for params in all_params
    data, _ = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_single_experiment;
        prefix = "scan",
        tag = false,
        verbose = false
    )

    label_text = params[:model_type] == :base ?
        "base, a=$(params[:a])" :
        "extended, k=$(params[:k])"

    plot!(
        p6,
        data["t_points"], data["y_values"],
        label = label_text,
        lw = 2,
        alpha = 0.8
    )
end
plot!(
    p6,
    xlabel = "t",
```

```
    ylabel = "y(t)",
    title = "Сканирование: траектории y(t) для разных параметров",
    legend = :outerright,
    grid = true
)
savefig(p6, plotsdir(script_name, "parametric_scan_y_comparison.png"))
```

5.15 Сравнительный фазовый портрет

```
p7 = plot(size = (950, 520), dpi = 150)
for params in all_params
    data, _ = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_single_experiment;
        prefix = "scan",
        tag = false,
        verbose = false
    )

    label_text = params[:model_type] == :base ?
        "base, a=$(params[:a])" :
        "extended, k=$(params[:k])"

    plot!(
        p7,
        data["x_values"], data["y_values"],
```

```
        label = label_text,
        lw = 2,
        alpha = 0.8
    )
end
plot!(
    p7,
    xlabel = "x",
    ylabel = "y",
    title = "Сканирование: фазовые траектории",
    legend = :outerright,
    grid = true
)
savefig(p7, plotsdir(script_name, "parametric_scan_phase_comparison.png"))
```

5.16 График зависимости norm_final от параметров

```
p8 = plot(size = (900, 520), dpi = 150)

base_sub = results_df[results_df.model_type .== :base, :]
extended_sub = results_df[results_df.model_type .== :extended, :]

if nrow(base_sub) > 0
    plot!(
        p8,
        base_sub.a, base_sub.norm_final,
        seriestype = :scatter,
```

```
        label = "base"
    )
end

if nrow(extended_sub) > 0
    plot!(
        p8,
        extended_sub.k, extended_sub.norm_final,
        seriestype = :scatter,
        label = "extended"
    )
end

plot!(
    p8,
    xlabel = "Параметр модели",
    ylabel = "norm_final",
    title = "Зависимость norm_final от параметров модели",
    legend = :topleft,
    grid = true
)
savefig(p8, plotsdir(script_name, "norm_final_vs_parameter.png"))
```

5.17 Бенчмаркинг

```
println("\n" * "="^60)
println("БЕНЧМАРКИНГ ДЛЯ РАЗНЫХ ПАРАМЕТРОВ")
```

```
println("="^60)

benchmark_results = []

for params in all_params
    function benchmark_run()
        u0 = [params[:x0], params[:y0]]

        if params[:model_type] == :base
            p = (a = params[:a], b = params[:b], c = params[:c], d = params[:d])
            prob = ODEProblem(syst_base!, u0, params[:tspan], p)
        else
            p = (a = params[:a], b = params[:b], c = params[:c], d = params[:d], k =
            prob = ODEProblem(syst_extended!, u0, params[:tspan], p)
        end

        return solve(
            prob,
            params[:solver];
            saveat = LinRange(params[:tspan][1], params[:tspan][2], params[:nt])
        )
    end

    println("\nБенчмарк для model_type=$(params[:model_type]), a=$(params[:a]), k=$(p
    bmark = @benchmark $benchmark_run() samples = 80 evals = 1
    tsec = median(bmark).time / 1e9
    println(" Медианное время: ", round(tsec; digits = 6), " сек")
```

```
    push!(benchmark_results, (
        model_type = string(params[:model_type]),
        a = params[:a],
        k = params[:k],
        time = tsec
    ))
end

bench_df = DataFrame(benchmark_results)
CSV.write(datadir(script_name, "benchmark_results.csv"), bench_df)
```

5.18 График времени вычисления

```
p9 = plot(size = (900, 520), dpi = 150)

bench_base = bench_df[bench_df.model_type .== "base", :]
bench_extended = bench_df[bench_df.model_type .== "extended", :]

if nrow(bench_base) > 0
    plot!(
        p9,
        bench_base.a, bench_base.time,
        seriestype = :scatter,
        label = "base"
    )
end
```

```
if nrow(bench_extended) > 0
    plot!(
        p9,
        bench_extended.k, bench_extended.time,
        seriestype = :scatter,
        label = "extended"
    )
end

plot!(
    p9,
    xlabel = "Параметр модели",
    ylabel = "Время вычисления, сек",
    title = "Зависимость времени решения ODE от параметров модели",
    legend = :topleft,
    grid = true
)
savefig(p9, plotsdir(script_name, "computation_time_vs_parameter.png"))
```

5.19 Сохранение всех результатов

```
@save datadir(script_name, "all_results.jld2") base_params base_params2 param_grid al
@save datadir(script_name, "all_plots.jld2") p1 p2 p3 p4 p5 p6 p7 p8 p9

println("\n" * "="^60)
println("ЛАБОРАТОРНАЯ РАБОТА ЗАВЕРШЕНА")
println("="^60)
```

```
println("\nРезультаты сохранены в:")
println(" • data/$(script_name)/single/ - базовые эксперименты")
println(" • data/$(script_name)/parametric_scan/ - параметрическое сканирование")
println(" • data/$(script_name)/results_summary.csv - сводная таблица")
println(" • data/$(script_name)/benchmark_results.csv - результаты бенчмаркинга")
println(" • data/$(script_name)/all_results.jld2 - сводные данные")
println(" • plots/$(script_name)/ - все графики")
println(" • data/$(script_name)/all_plots.jld2 - объекты графиков")
```

5.20 Базовые эксперименты

5.20.1 Базовая модель (model_type = base)

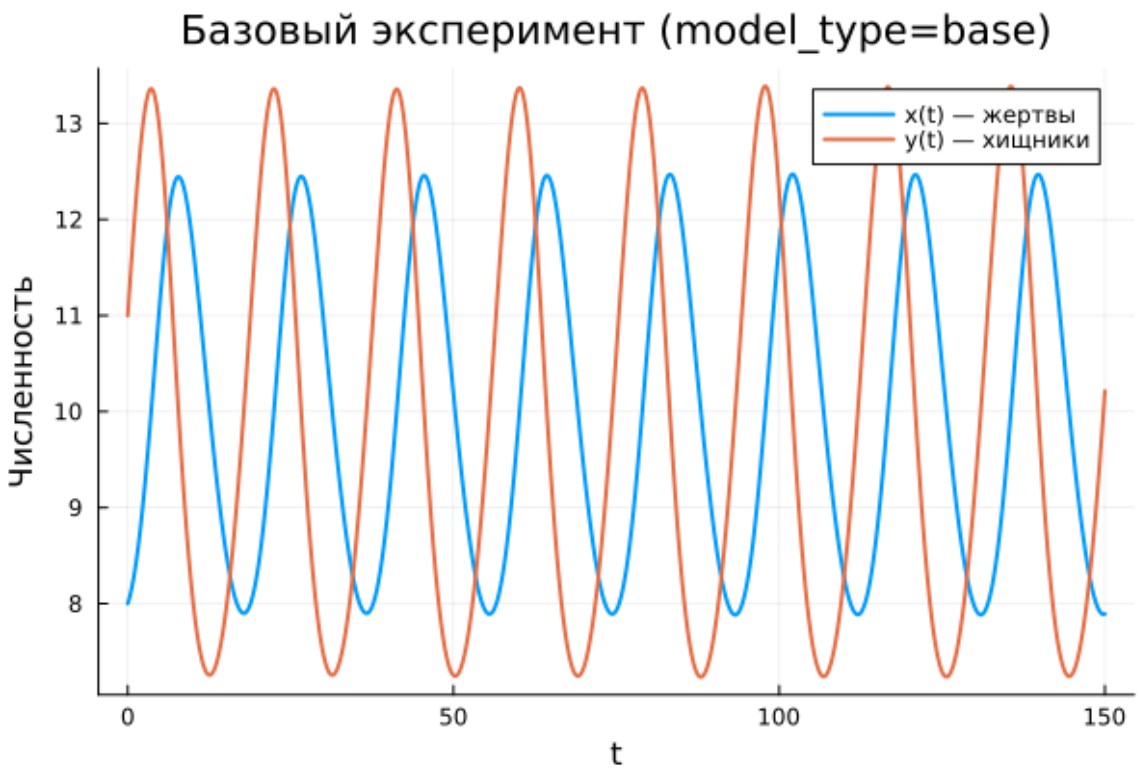

Рисунок 5.1: Базовая модель: зависимость x(t), y(t)

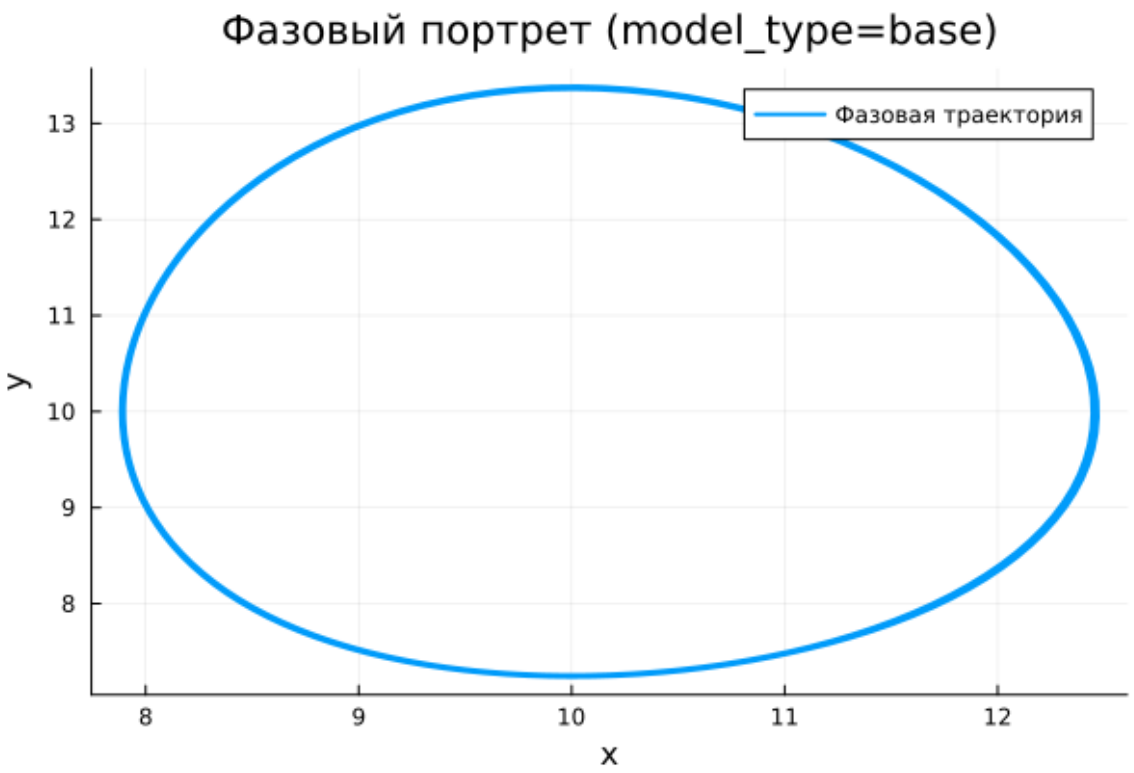

Рисунок 5.2: Базовая модель: фазовый портрет

Для базовой модели на графиках временных зависимостей наблюдаются устойчивые периодические колебания обеих переменных. Численности жертв $x(t)$ и хищников $y(t)$ изменяются во времени регулярно, причём амплитуда колебаний на всём интервале остаётся практически постоянной.

Такое поведение соответствует классической колебательной динамике системы хищник–жертва без затухания. Популяции не переходят к стационарному состоянию и не демонстрируют затухания амплитуды, а продолжают совершать повторяющиеся колебания около некоторого среднего уровня.

Фазовый портрет базовой модели имеет вид замкнутой овальной траектории. Это подтверждает периодический характер движения: система многократно возвращается в те же фазовые состояния, а траектория остаётся ограниченной.

5.20.2 Расширенная модель (model_type = extended)

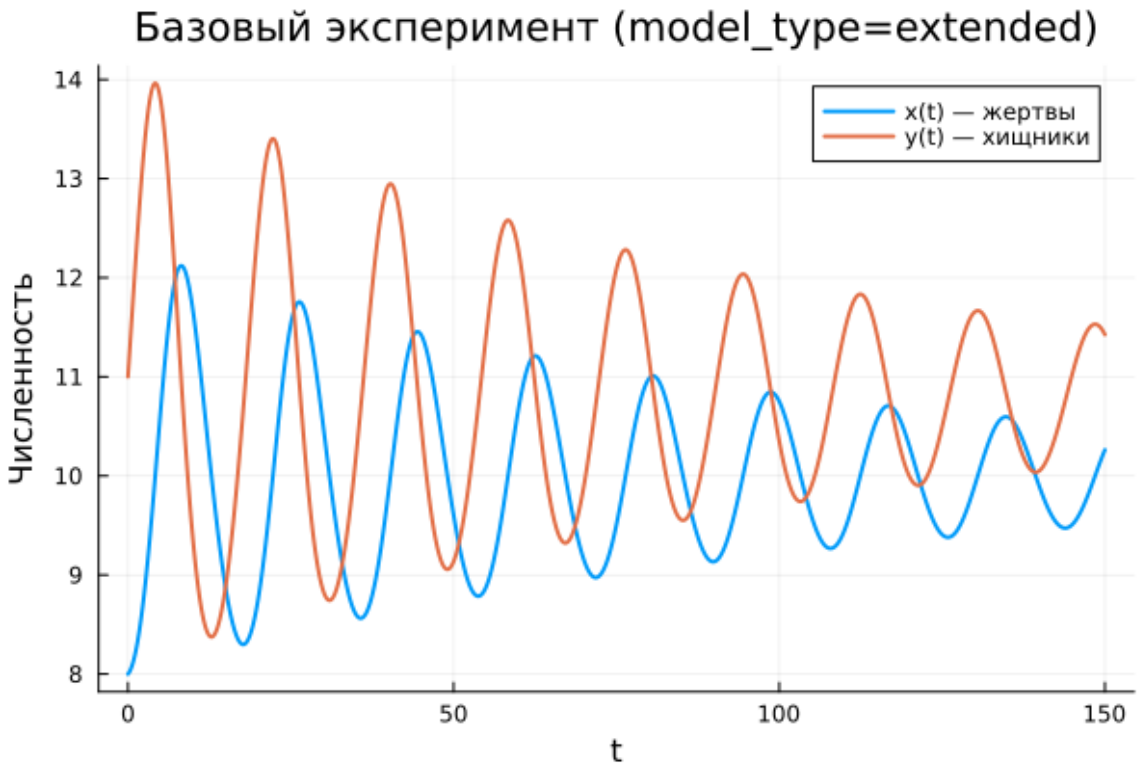

Рисунок 5.3: Расширенная модель: зависимость x(t), y(t)

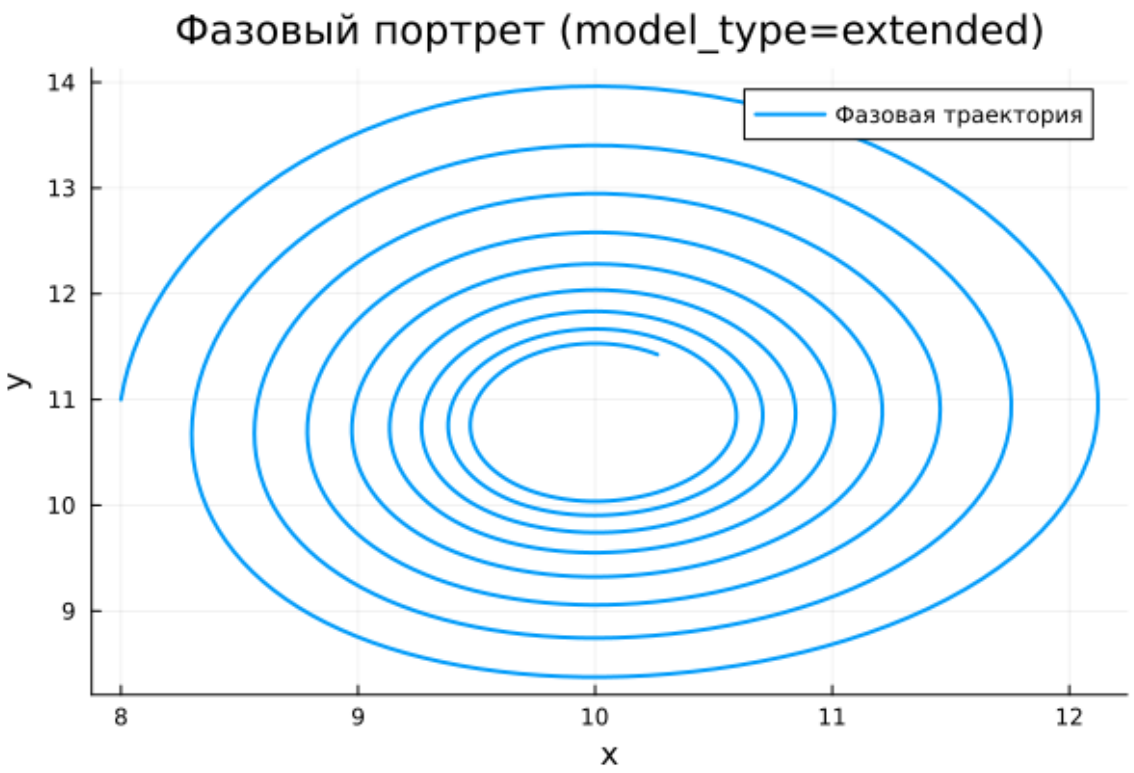

Рисунок 5.4: Расширенная модель: фазовый портрет

В расширенной модели также наблюдается колебательный процесс, однако его характер существенно отличается от базового случая. На графиках $x(t)$ и $y(t)$ видно, что в начале движения амплитуда колебаний достаточно велика, но затем постепенно уменьшается.

Это связано с появлением дополнительного нелинейного члена $-kx^2$, который ограничивает рост популяции жертв. В результате система теряет исходную консервативность, и колебания становятся затухающими по амплитуде. Со временем решение стремится не к нулю, а к некоторому устойчивому стационарному режиму.

Фазовый портрет расширенной модели подтверждает этот вывод. Траектория имеет вид закручивающейся спирали, которая постепенно стягивается к внутренней области фазового пространства. Это означает, что после переходного процесса система приближается к устойчивому состоянию равновесия.

Следовательно, расширенная модель демонстрирует затухающие колебания с выходом на стационарное состояние. Дополнительный нелинейный механизм стабилизирует динамику и подавляет незатухающие циклы, характерные для базовой модели.

5.21 Параметрическое сканирование

5.21.1 Траектории $x(t)$ для различных параметров

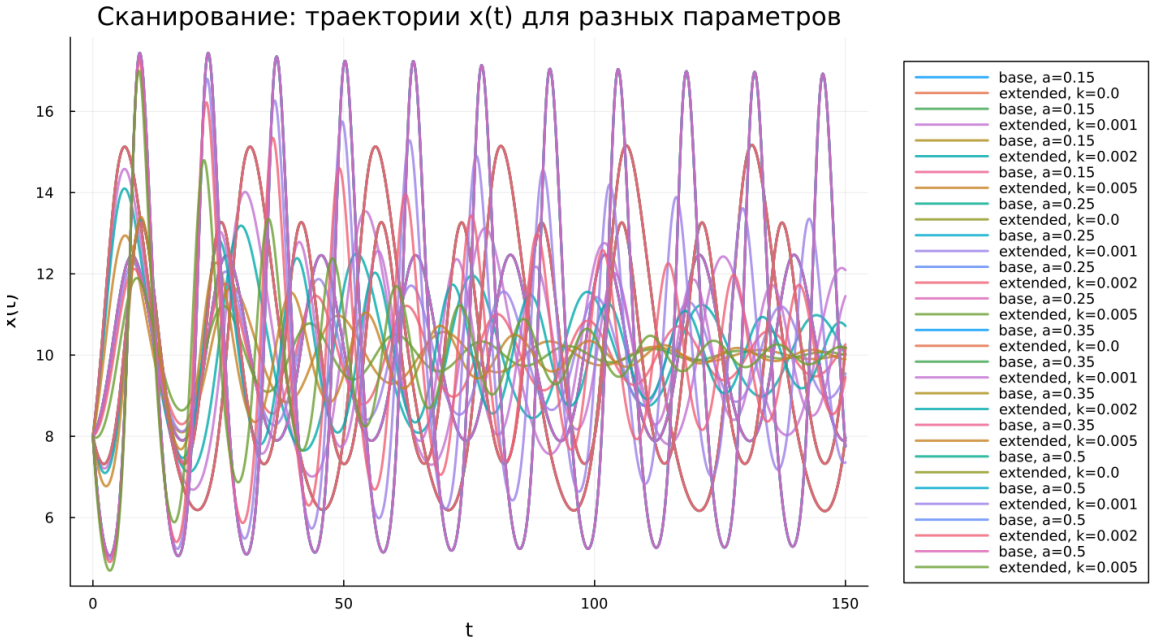

Рисунок 5.5: Сканирование траекторий x(t)

Было проведено параметрическое исследование двух моделей. Для базовой модели варьировался параметр a, а для расширенной — параметр k, отвечающий за дополнительное ограничение роста популяции жертв.

Для базовой модели изменение параметра a влияет прежде всего на характер периодических колебаний. При разных значениях параметра изменяются амплитуда и частота колебаний, однако общий режим остаётся колебательным:

траектории не затухают и сохраняют выраженную периодичность на всём временном интервале.

Для расширенной модели изменение параметра k существенно влияет на скорость затухания и уровень, к которому стремится решение. При увеличении k вклад нелинейного ограничения усиливается, вследствие чего колебания быстрее подавляются, а переход к стационарному режиму происходит более заметно.

Основные наблюдения:

- базовая модель сохраняет устойчивый колебательный режим;
- расширенная модель демонстрирует затухание колебаний;
- параметры влияют на амплитуду, частоту и скорость установления решения.

5.21.2 Траектории $y(t)$ для различных параметров

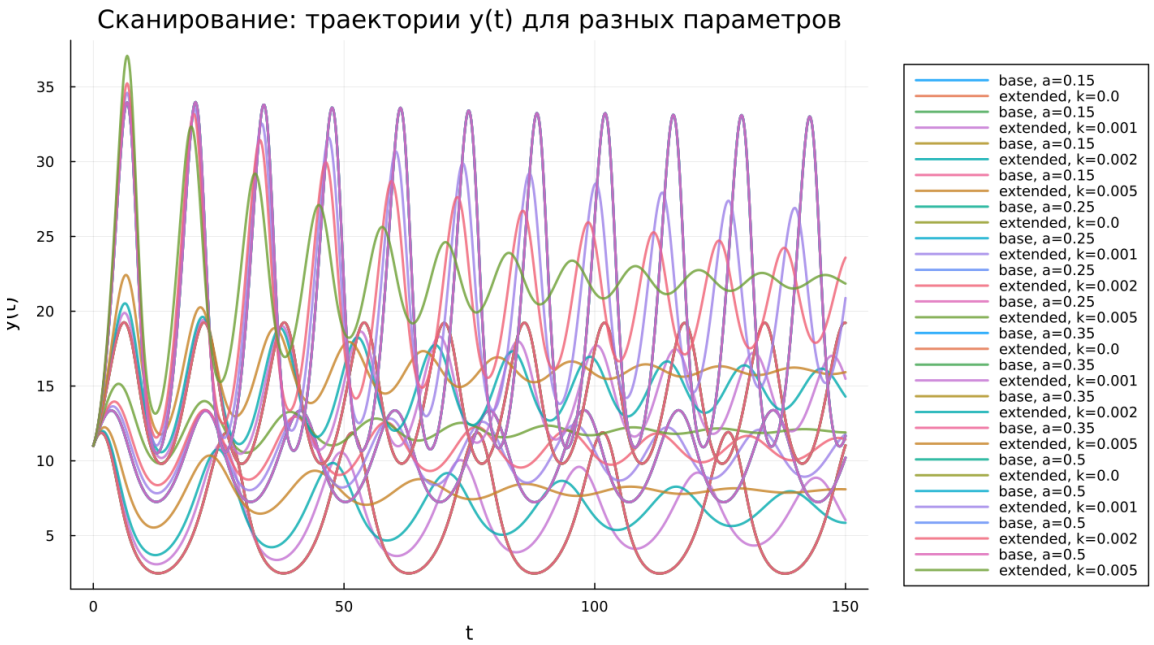

Рисунок 5.6: Сканирование траекторий y(t)

Анализ траекторий $y(t)$ показывает аналогичные закономерности. Для базовой модели переменная сохраняет регулярные периодические колебания, причём изменение параметра a приводит к изменению формы и масштаба этих колебаний, но не разрушает сам периодический режим.

В расширенной модели семейство кривых демонстрирует постепенное уменьшение амплитуды и выход на различные стационарные уровни. Чем сильнее нелинейное ограничение, тем быстрее исчезают большие колебания и тем быстрее система приближается к устойчивому состоянию.

5.21.3 Фазовые траектории для различных параметров

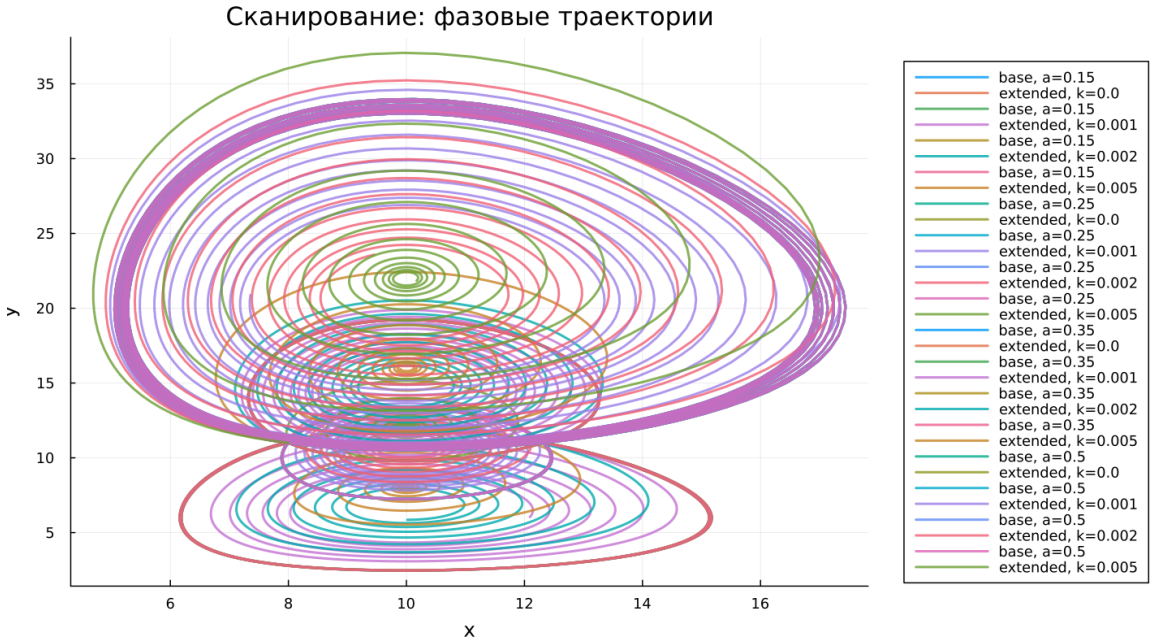

Рисунок 5.7: Сканирование фазовых траекторий

Сравнение фазовых траекторий позволяет наглядно увидеть различие в структуре динамики при разных параметрах и для разных типов модели.

Для базовой модели фазовые траектории в основном представляют собой замкнутые кривые различного размера и формы. Это указывает на сохранение периодического режима и отсутствие стремления к единственной точке

равновесия.

Для расширенной модели фазовые траектории имеют спиралевидный характер и постепенно стягиваются к внутренним областям фазового пространства. Такое поведение соответствует затухающим колебаниям и наличию устойчивого состояния, к которому стремится система после переходного процесса.

Следовательно, фазовые портреты подтверждают, что параметрическое изменение коэффициентов не меняет фундаментальную природу моделей: базовая система остаётся автоколебательной, а расширенная — стабилизирующейся.

5.22 Анализ метрики norm_final

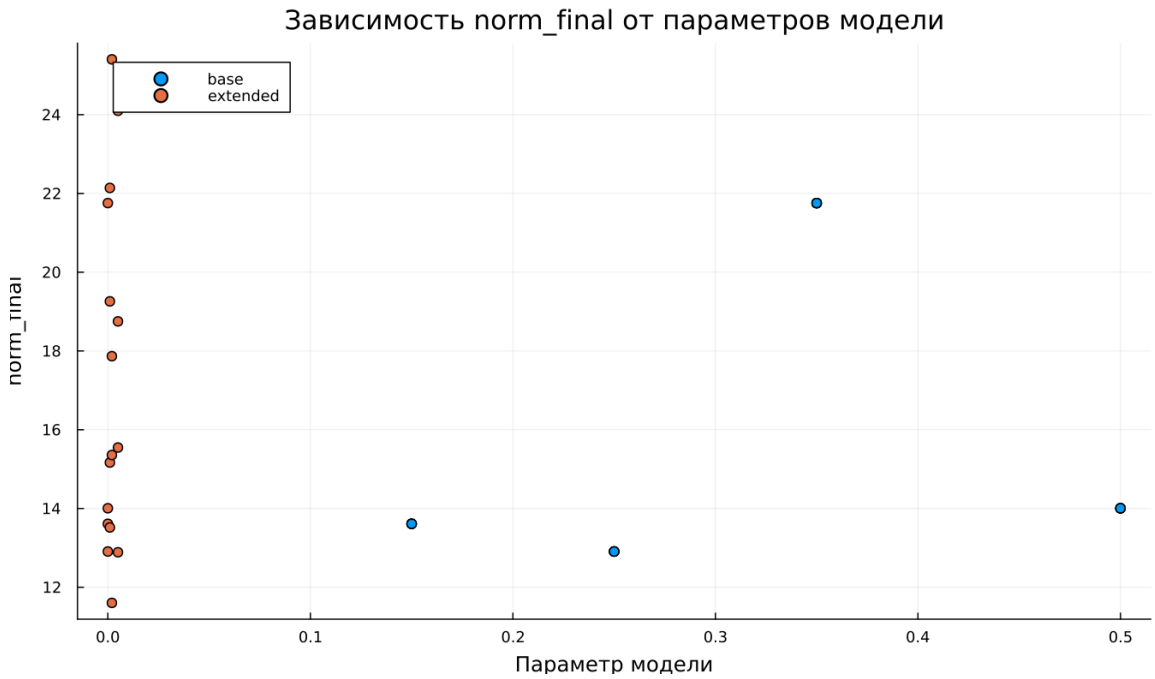

Рисунок 5.8: Зависимость norm_final

Рассматривалась метрика

$$\text{norm_final} = \sqrt{x(t_{final})^2 + y(t_{final})^2}.$$

Полученные результаты показывают, что для базовой модели значение norm_final остаётся сравнительно большим и заметно зависит от параметра a. Это связано с тем, что к моменту окончания интегрирования система продолжает находиться в колебательном режиме и не переходит к стационарному состоянию.

Для расширенной модели значения метрики также зависят от параметра k, однако их интерпретация иная. Здесь величина norm_final определяется уже не фазой незатухающего цикла, а положением устойчивого состояния, к которому система приближается после затухания переходных колебаний.

5.23 Время вычислений

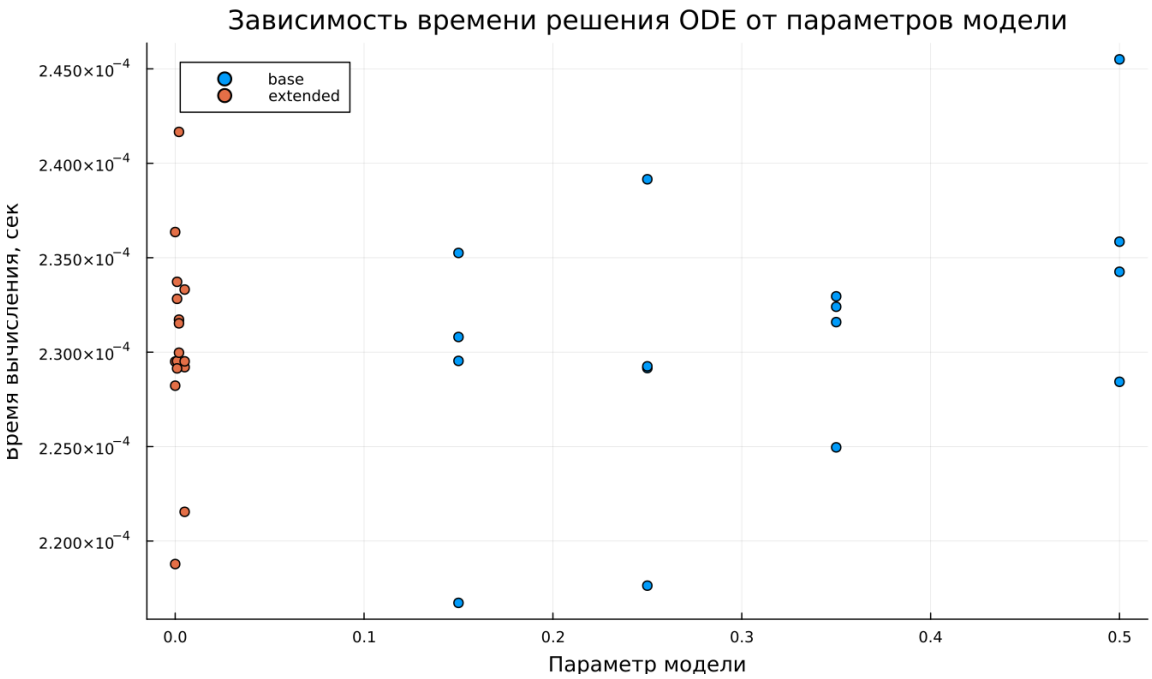

Рисунок 5.9: Зависимость времени вычисления

Бенчмаркинг показывает, что время численного решения во всех экспериментах остаётся очень малым и имеет один порядок величины. Это свидетельствует о хорошей вычислительной эффективности используемой схемы

интегрирования для обеих моделей.

Для базовой модели наблюдается небольшой разброс времени вычисления при изменении параметра a. Для расширенной модели время также меняется несущественно при варьировании параметра k.

Можно отметить, что расширение модели дополнительным нелинейным членом не приводит к резкому росту вычислительных затрат. Следовательно, усложнение структуры системы практически не ухудшает её численную разрешимость на выбранном интервале времени.

5.24 Выводы

1. Базовая модель демонстрирует устойчивые незатухающие периодические колебания.
2. Расширенная модель характеризуется затухающими колебаниями и выходом на стационарное состояние.
3. Фазовые портреты подтверждают различие между замкнутыми периодическими траекториями базовой модели и спиральным стягиванием в расширенной модели.
4. Параметры a и k влияют на амплитуду, частоту и скорость установления решения.
5. Метрика norm_final отражает качественное различие между незатухающим и стабилизирующимся режимами.
6. Обе модели эффективно вычисляются численно, а увеличение сложности системы не приводит к существенному росту времени расчёта.

Список литературы

1. Модель Лотки-Вольтерры
2. Lotka-Volterra System